

LeetCode

746. 使用最小花费爬楼梯

给定一个整数数组 `cost`，其中 `cost[i]` 是从楼梯第 *i* 个台阶向上爬需要支付的费用。一旦你支付此费用，即可选择向上爬一个台阶或者两个台阶。
返回达到楼梯顶部的最低花费。这里的顶部指不在 `cost` 数组范围内的最后一个台阶。

```
class Solution {
public:
    int minCostClimbingStairs(vector<int> cost) {
        int n = cost.size();
        if (n == 0) return 0;
        if (n == 1) return cost[0];
        if (n == 2) return min(cost[0], cost[1]);
        int f0 = 0, f1 = cost[0], f2 = cost[1];
        for (int i = 2; i < n; i++) {
            f2 = min(f1 + cost[i], f0 + cost[i]);
            f0 = f1;
            f1 = f2;
        }
        return f2;
    }
};
```

198. 打家劫舍

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。
给定一个代表每个房屋存放金额的非负整数数组，计算你不触动警报装置的情况下，一夜之内能够偷窃到的最高金额。

示例 1:
输入: [1,2,3,1]
输出: 4
解释: 偷窃 1 号房屋 (金额 1) 和 3 号房屋 (金额 3)。偷窃的总金额 = 1 + 3 = 4。

示例 2:
输入: [2,7,9,3,1]
输出: 12
解释: 偷窃 1 号房屋 (金额 2)、3 号房屋 (金额 9) 和 5 号房屋 (金额 1)。偷窃的总金额 = 2 + 9 + 1 = 12。

```
class Solution {
public:
    int rob(vector<int> nums) {
        int n = nums.size();
        if (n == 0) return 0;
        if (n == 1) return nums[0];
        if (n == 2) return max(nums[0], nums[1]);
        int f0 = 0, f1 = nums[0], f2 = max(nums[0], nums[1]);
        for (int i = 2; i < n; i++) {
            f2 = max(f1 + nums[i], f0 + nums[i]);
            f0 = f1;
            f1 = f2;
        }
        return f2;
    }
};
```

118. 杨辉三角

给定一个非负整数 `numRows`，生成杨辉三角的前 `numRows` 行。
在杨辉三角中，每个数是它左上方和右上方的数的和。

示例:
输入: 5
输出:
[[1],
[1,1],
[1,2,1],
[1,3,3,1],
[1,4,6,4,1]]

```
class Solution {
public:
    vector<vector<int>> generate(int numRows) {
        vector<vector<int>> triangle;
        for (int i = 0; i < numRows; i++) {
            vector<int> row(i + 1, 1);
            for (int j = 1; j < i; j++) {
                row[j] = triangle[i-1][j-1] + triangle[i-1][j];
            }
            triangle.push_back(row);
        }
        return triangle;
    }
};
```

94. 二叉树的中序遍历

给定一个二叉树，返回它的中序遍历。
进阶: 你不得递归地解决这个问题。

示例:
输入: [1,2,3,4,5,6,7]
输出: [2,1,4,3,6,5,7]

```
class Solution {
public:
    vector<int> inorderTraversal(TreeNode* root) {
        vector<int> result;
        if (root == NULL) return result;
        inorderTraversal(root->left);
        result.push_back(root->val);
        inorderTraversal(root->right);
        return result;
    }
};
```

119. 杨辉三角 II

给定一个非负整数 `rowIndex`，返回杨辉三角中指定行的元素。
在杨辉三角中，每个数是它左上方和右上方的数的和。

示例:
输入: 3
输出: [1,3,3,1]

```
class Solution {
public:
    vector<int> getRow(int rowIndex) {
        vector<int> row(rowIndex + 1, 1);
        for (int i = 1; i < rowIndex; i++) {
            for (int j = i; j > 0; j--) {
                row[j] = row[j-1] + row[j];
            }
        }
        return row;
    }
};
```